ChatGPT

Room XI Connect App Code Review and Quality Assurance Report

Overview of the Application

Room XI Connect is a youth-focused web application developed with **Vite**, **React** (TypeScript) on the front-end, and **Supabase** on the back-end for authentication, storage, and PostgreSQL database. The styling is done with **Tailwind CSS**, and the app is configured as a Progressive Web App (PWA) with offline support. The primary goal is to help youth discover local programs, track mood check-ins, manage consents, and facilitate attendance tracking at in-person events. An admin dashboard is provided for system monitoring and audit logs.

The codebase is organized for clarity and maintainability into distinct folders:

- `src/routes` – React page components for main views (Explore, Home, Me/Profile, ProgramDetail, SafetyProfile, Settings, Admin, and auth-related screens like Register/Login/Reset).
- `src/ui` – Reusable UI components and design elements (forms, buttons, navigation, etc., including subfolders like `explore` for program listing UI).
- `src/lib` – Utility modules (Supabase client setup, session management, QR code handling, offline queue, encryption utilities).
- `src/ximi` – Logic for the AI assistant (e.g. a heuristic chat analysis module).
- `public` – Static assets, including the service worker `sw.js` for offline caching and manifest for PWA.
- `supabase/` – Database schema, seed data, row-level security (RLS) policies, and Edge Functions (like `xid-create` for generating anonymous IDs and `user-delete` for account deletion).

Overall, the project follows good practices with separation of concerns, making the codebase relatively easy to navigate and maintain. TypeScript types are used throughout for clarity. The use of Supabase and RLS means most data enforcement happens server-side, adding security but requiring careful sync between front-end and DB schema.

Summary of Major Features and Workflows

Feature / Workflow	Description
Authentication & Onboarding	Users can register via email/password, verify their email, and complete a multi-step sign-up form collecting basic details (name, preferred name, age, location, and guardian info if under 18). Users must consent to terms, privacy policy, and data collection, which are recorded in the database. Email verification links are sent via Supabase.

Feature / Workflow	Description
Program Discovery (Explore)	The Explore tab lists available youth programs with options to filter by category/tags, view programs on a map, and toggle between list or map view. Users can click on a program to see details and can **save/unsave** programs to their profile for later. The Explore page also includes a chat **AI assistant (Ximi)** dock that can answer questions and provide guidance (e.g. suggest programs or detect crisis keywords).
Home & Mood Tracking	The Home screen allows users to check in daily with a mood rating and optionally tag feelings or leave notes. Mood history is charted and a streak counter is shown to encourage regular check-ins. Based on mood or interests, the Home screen can suggest relevant programs or resources.
Safety Profile & Consents	Users maintain a safety profile with emergency contact info, medical or health notes, and optional self-identification (e.g., Indigenous identity). The app also collects fine-grained consents (photo consent, data sharing consent, etc.). Changes to consents are logged to an audit table for accountability. If the user is under 18, the app can initiate guardian consent verification (e.g., sending an email to a parent/guardian for approval).
Attendance & XID System	Each user is assigned a pseudo-anonymous **XID** (external ID) which is a hash used to check in at in-person events without revealing personal info. Users can scan a QR code at an event or manually enter a code to record attendance. Attendance records are stored and can sync while offline via an encrypted queue (the data is encrypted on-device with a secret key and sent to the server when connectivity is restored).
Admin Dashboard	An admin-only section shows aggregate stats (counts of users, mood check-ins, programs, attendance records) and a list of recent audit log entries for administrative actions (e.g., any manual data edits or consent overrides). Only users flagged as admins (determined via a Supabase function or role) can access this panel.
Settings & Account Management	The Settings page allows users to manage their account: sign out, initiate account deletion (which calls a Supabase Edge Function to sanitize and remove personal data), and toggle preferences. Currently a dark mode toggle is present (labelled "coming soon") and a notifications toggle (UI only). There is also a placeholder to **export user data** (for GDPR/PHIPA compliance) not yet fully implemented. The app can be installed as a PWA from this page.
Offline Support (PWA)	The app registers a service worker (`public/sw.js`) to cache assets and enable offline usage. Certain actions, like program save/unsave or event check-in, will be queued locally if the device is offline. The queue system (in `src/lib/queue.ts`) encrypts each action with AES-GCM using a secret key, and periodically retries to sync with the backend when back online. This ensures the app is functional even with intermittent connectivity.

Critical Issues Found and Fixes Applied

During the review, several bugs and inconsistencies were identified in the code. These would have caused runtime errors or incorrect behavior. The following table lists the critical issues found and the fixes that have been applied to the codebase (in the latest source code), along with notes for further improvement:

File/Component	Issue	Fix Applied	Notes
`src/routes/Admin.tsx`	Admin status was determined by selecting a non-existent `role` column from the `profiles` table, and audit logs were queried from an `audit_logs` table that does not exist in the schema. The UI expected fields `old_values` / `new_values`, but the actual audit log uses different field names. This led to admin panel failing to load.	Used the existing helper `isUserAdmin` (which calls a Supabase RPC `is_admin`) to check admin privileges instead of a nonexistent `role`. Updated the log query to use the correct `admin_logs` table (limited to recent 50 entries). Replaced references to `old_values` / `new_values` with `old_record` / `new_record` to match the actual table schema. Adjusted the display to show these JSON fields in a readable way.	The admin panel now loads correctly. In the future, the admin logs view could be enhanced with filters (by table or date) and possibly an export feature for analysis.
`src/lib/attendance.ts` (functions `getAttendanceHistory` & `getAttendanceCount`)	These functions attempted to query attendance by passing a Supabase query builder object into `.in('xid_id', …)` instead of an array of values. This is a logical error (`.in` expects a list) and resulted in empty results or thrown errors, meaning user attendance history/count would not display.	Refactored these functions to first fetch the list of the user's XID values from the `xids` table, then supply that array to the `.in('xid_id', [...])` filter. Added error handling in case the XID lookup fails.	With this fix, the Home/Profile can correctly show the user's past attendance records and count. Ensure to test with a user who has multiple XIDs (if the system rotates IDs) to verify all attendance records are fetched.

File/Component	Issue	Fix Applied	Notes
`src/routes/auth/UpdatePassword.tsx`	The password reset page attempted to destructure `supabase.auth.getSession()` immediately (synchronously), but `getSession()` returns a Promise. As a result, the code always got `undefined` for the session and immediately redirected the user to login, making the password reset link flow broken.	Changed the logic to use an `useEffect` hook that calls `getSession().then(...)`. It only triggers a redirect if no session is found *after* the promise resolves. Also added a cleanup to avoid state updates on unmounted component.	Now, when a user follows an email reset link to this page, they can actually set a new password without being kicked out. It's recommended to test this flow in multiple browsers to ensure the timing is reliable.
`src/routes/Admin.tsx` (UI fields)	Even after fixing the admin log query, the UI was still referencing `log.old_values` and `log.new_values` in some places, which would be `undefined`.	Updated all references to use `log.old_record` / `log.new_record` consistently when rendering admin log entries.	This was a minor follow-up fix to ensure no console errors and that the admin log JSON is properly displayed.

Note: These critical fixes have been applied to the code. It's important to re-deploy the updated code and run through the affected features (admin panel, attendance history, password reset) to confirm that the issues are resolved in the staging environment.

Remaining Functional Issues & Recommendations

In addition to the above critical fixes, the review identified several functional gaps, minor bugs, or areas for improvement. Addressing the following items will help ensure the app is smooth and fully ready for real users:

1. **Dark Mode & Notifications:**
2. The Settings page includes a dark mode toggle labeled "Coming soon" and a notifications toggle that currently only updates local state (no real push notification logic). These features are not yet functional. **Recommendation:** Implement dark theme support (e.g., using Tailwind's dark mode classes and a theme context) so that the toggle actually switches the UI to a dark palette. For

notifications, integrate with the Service Worker or a Push Notifications API to allow daily reminders for mood check-ins (if desired), or remove the toggle until implemented to avoid confusion.

3. **Program Schema vs UI Mismatch:**

4. The `ProgramDetail.tsx` component expects fields like `long_description`, `contact_email`, `contact_phone`, `website_url`, `capacity`, `age_min`, `age_max` which are **not present** in the `programs` table as defined (the provided schema has fields such as `title`, `description` (short description), `tags`, `free` (boolean), `indoor`/`outdoor`, `cost_cents`, `location_name`, `address`, `lat/lng`, `organizer`, `accessibility_notes`, `next_start`, `next_end`). This mismatch means those extra details will show as blank in the UI. **Recommendation:** To prevent undefined data in program details, either extend the Supabase `programs` table to include these fields and ensure they are populated, **or** simplify the front-end to use the fields that do exist (for example, use the `description` field in place of a missing `long_description`, remove or hide contact info fields if not available). Aligning the UI with the actual data schema is crucial for a polished user experience.

5. **Admin Log Export & Filtering (Unimplemented Features):**

6. In the Admin panel, there are UI controls for exporting logs (perhaps to CSV/Excel) and filtering logs by table or date, but these controls are not wired up. Clicking them currently does nothing. **Recommendation:** Either implement these features or hide/disable them for the pilot. For export, you could use a library or Supabase function to generate a CSV of `admin_logs`. For filtering, add query parameters to the log fetch (e.g., filter by table name or time range). Since these are admin tools, it's acceptable to launch without them as long as the absence is clear to the user.

7. **Password Reset Success Redirect:**

8. After successfully updating the password on the UpdatePassword page, the UI shows a success message and then uses a `setTimeout` to redirect the user to the login screen after 3 seconds. While this works, it could be prone to timing issues or might not be obvious to all users. **Recommendation:** Test this on various browsers to ensure the redirect consistently happens after showing the message. Additionally, consider providing an immediate "Back to Sign In" button or link upon success, so users have a clear action instead of waiting, improving the UX.

9. **Offline Queue & Encryption Key:**

10. The offline queue system (for saving actions when offline) uses AES-GCM encryption. It requires a secret key (`CRYPTO_KEY`) to be set in the environment. If this is not configured in production, the queue will not function (encryption calls will fail or data won't be decryptable on the server). **Recommendation:** Make sure to set a strong `CRYPTO_KEY` in production environment variables. Test the offline functionality by performing actions like adding a mood or checking into a program while offline (use dev tools to simulate offline), then coming online — those actions should sync correctly without data loss. Also handle cases where the encryption might fail (e.g., catch errors and alert the user or log for debugging).

11. **Edge Function Secrets & Permissions:**

12. The app relies on Supabase Edge Functions (`xid-create` to generate a new XID and `user-delete` to scrub user data on account deletion). These functions require certain secrets: `SUPABASE_URL`, `SUPABASE_ANON_KEY` (for service calls), `SUPABASE_SERVICE_ROLE_KEY` (for privileged DB access), and `XID_HASH_SECRET` (to hash/encode the XID). **Recommendation:** Verify that in the Supabase project settings and the deployment (e.g., Vercel or wherever the functions run) these environment variables are set. Without them, those functions will fail (for example, account deletion would not actually remove data, or XID generation might not work). Also, review the RLS policies to ensure that even if a user figures out the endpoints, they cannot misuse them (Supabase Edge Functions run with service role by default, so use checks inside the function as needed to ensure the caller is authorized to perform the action).

13. **Signup Guardian Verification Process:**

14. During registration, if the user is under 18, the form collects guardian contact info and is supposed to send a verification request (to guardian's email or phone). Currently, the implementation always chooses `verification_method: 'email_link'` (likely via an email sent through the Supabase Edge Function or a third-party email service). If a phone number is provided, there isn't an SMS verification implemented. **Recommendation:** If you need to support phone/SMS for guardian consent, integrate an SMS provider (e.g., Twilio) or configure a Supabase function to handle SMS. At minimum, make it clear in the UI that guardian verification will be via email (so users provide an email). Also test the guardian consent flow: when a minor signs up, ensure an email is actually sent to the guardian with a working verification link, and that verifying it properly marks the youth account as approved in the database.

15. **Data Export & Privacy Compliance:**

16. The Settings page has a placeholder for "Export my data". Currently, clicking this doesn't trigger a real download. In many jurisdictions (GDPR, PHIPA, etc.), users have the right to obtain a copy of their data. **Recommendation:** Implement a backend endpoint or function that gathers all of a user's data (profile, mood entries, program saves, attendance records, consents, etc.) into a JSON or CSV file. This could be done via a Supabase Cloud Function or a secure server endpoint that queries the database and archives the data. Since this can be a heavy operation, you might implement it as an async request where the user gets an email with a download link when ready. If this feature will not be ready for the pilot, remove or hide the option for now to avoid user frustration.

17. **Accessibility (WCAG) & Localization:**

18. The app should be reviewed for accessibility compliance (WCAG 2.1 AA standards). Potential issues to check: proper use of semantic HTML in components, sufficient color contrast (some Tailwind styling uses very light grays which may be low-contrast for text), focus order and focus visibility (especially with modals or slide-over panels), and the presence of appropriate ARIA labels for interactive elements (like icon buttons for closing modals, etc.). During testing, use tools like axe-core (which is already included as a dev dependency) to catch accessibility issues. Additionally, ensure that screen reader behavior is considered (for example, when new content loads or when error messages appear, they should be announced). The codebase also has i18n support (with i18next); verify that

text is properly externalized for localization and that the default language is set appropriately. These steps will make the app more usable to a wider range of users and meet compliance requirements.

19. **Rate Limiting & Security Hardening:**

    - Security is critical since the app deals with youth data. Currently, the Express server (if any) and Supabase should have protections in place, but a few considerations: Ensure RLS (Row Level Security) is enabled on all tables in Supabase with policies that only allow each user to access their own data (the provided `policies.sql` should cover this). The use of an `is_admin` RPC suggests that admin rights are granted via a separate mechanism; ensure a malicious user cannot simply toggle a flag in their JWT to become admin. Also, consider enabling rate limiting on critical routes like login and registration (there is a dependency `express-rate-limit`, indicating this might be in use on the server). Verify that the service worker and any external scripts are served over HTTPS and that a strict Content Security Policy (CSP) is in place (the `DEPLOYMENT.md` likely has a recommended CSP). All third-party integrations (maps, OpenAI, etc.) should use keys that are restricted to your domain. Finally, test for common vulnerabilities (XSS, SQL injection – though Supabase parameterizes queries – and ensure no sensitive info is exposed in client-side code or error messages).

20. **AI Assistant (Ximi) Integration:**

    - The Explore page includes an AI chat assistant ("Ximi") which can answer user questions and detect if a user's messages indicate a crisis. Currently, this appears to be implemented with a local heuristic (rule-based keywords in `src/ximi/heuristic.ts`) to categorize messages, and dependencies for OpenAI and Google's PaLM APIs are included, suggesting future enhancements. **Recommendation:** Decide on the scope of this assistant for the pilot. If keeping it active, ensure that it's clearly explained to users what it can and cannot do (since a rule-based bot might give limited responses). If integrating with an AI API for more sophisticated answers, verify that API keys are properly set in the environment and that using the AI respects privacy (don't send personal data in prompts, etc.). Test the assistant thoroughly: ask it general questions, as well as potential crisis phrases (the code likely flags words related to self-harm or abuse to prompt a "crisis resources" response). Make sure it responds appropriately and provides the user with helpful info or links to crisis support if needed. If this feature isn't fully ready or accurate, consider disabling it for now or labeling it as beta. It's a great idea, but it must be reliable given the sensitive context.

21. **Mobile Responsiveness & Device Testing:**

    - Although the app is a web PWA, it's expected that many users will access it on mobile devices. The code includes Capacitor setup scripts, indicating the app can be compiled into a native app (for iOS/Android). **Recommendation:** Test the UI on various device sizes (small smartphones, tablets). Ensure that all pages (especially forms and tables like the admin logs, or multi-column layouts like program list vs map) gracefully adapt to smaller screens. The map and QR code scanner should be checked on mobile: e.g., does the QR scanner access the camera properly via the browser or Capacitor plugin? Are touch targets (buttons, links) adequately large for mobile? Also test the PWA installation on Android/iOS – when installed, does it launch and function without issues (any missing meta tags or offline assets)? If using Capacitor to publish in app stores, ensure things like the splash screen, icons, and necessary

native permissions (camera for QR scanning, push notifications for future features) are configured. Verifying these will prevent any last-minute surprises when users try the app on their phones.

Each of the above points should be addressed or at least documented before the public pilot. By resolving these, we eliminate both critical blockers and papercuts that could frustrate users or admins.

Environment & Deployment Checklist

To ensure the application runs correctly in a staging or production environment, go through the following deployment checklist:

- **Supabase Database Setup:** Apply all database migrations and policies in your Supabase instance. This includes creating tables and functions as per the schema (if provided separately, e.g. in `schema.sql`), and setting up Row Level Security (RLS) policies from `policies.sql`. Also run the seed scripts for initial data: for example, seed the programs table with sample programs (`seed_programs.sql` or equivalent files in the repo) and any support resources (like crisis lines, etc., if provided).

- **Supabase Edge Functions:** Build and deploy the edge functions. Specifically, `xid-create` (generates a new hashed XID for a user, probably called upon new account creation or as needed) and `user-delete` (sanitizes and deletes a user's data for account deletion) need to be deployed via the Supabase CLI. After deploying, set the required environment variables in the Supabase project for these functions (e.g., `SUPABASE_URL`, `SUPABASE_ANON_KEY`, `SUPABASE_SERVICE_ROLE_KEY`, `XID_HASH_SECRET`) as mentioned earlier. Test the functions via Supabase's function tester or curl to ensure they respond as expected.

- **Environment Variables (.env):** On the front-end/Vercel (or whatever hosting), configure a `.env` or environment settings with:

- `VITE_SUPABASE_URL` and `VITE_SUPABASE_ANON_KEY` (for the Supabase client in the front-end).
- `CRYPTO_KEY` for offline queue encryption (ensure this is a 32-byte base64 or a strong key).
- API keys for any third-party services you plan to use in production (e.g., SendGrid API key for emails, an OpenAI API key if enabling the AI assistant's external calls, Google Maps API if map tiles require it, etc.).

- Any feature flags or secrets needed (for example, if using web push notifications, you'd need a VAPID key, etc.). Keep these values secret and do not commit them to the repo. Double-check that none of the keys or secrets are accidentally exposed in the client bundle (VITE_ prefixed keys will be embedded in front-end, which is expected for the anon key, but others like service_role or secret keys should only live in server-side config like Supabase functions).

- **Service Worker & PWA Configuration:** Ensure the `public/sw.js` is correctly referenced in your `index.html` and that the service worker registration is happening (usually in the app's initialization code). The service worker should cache essential assets and possibly some API responses for offline use. Since this is a pilot, consider how aggressive the caching should be (you might want to update the cache version in `sw.js` if you've made changes, so users get the latest

files). Also verify the PWA manifest (icons, name, theme color) is set for the pilot deployment. On iOS devices, since they have peculiar PWA support, test that as well.

- **Content Security Policy (CSP):** Set up a strong Content Security Policy on the hosting platform. At minimum, restrict scripts to only allow your own domain and known trusted sources. Supabase endpoints and any third-party integrations (like map tile servers or analytics) will need to be whitelisted. The `DEPLOYMENT.md` likely contains a sample CSP. Enforce HTTPS everywhere. Consider using security headers like X-Frame-Options, HSTS, etc., as recommended by OWASP for web apps, especially since this app deals with personal data.

- **Monitoring and Logging:** For the pilot, have monitoring in place. Supabase will log function calls and database errors; consider setting up alerts for errors. On the front-end, you might want a service like Sentry for catching any runtime exceptions that users encounter. Ensure that any console errors or warnings in the app are resolved or suppressed, so that the pilot testers (or their guardians) using browser dev tools won't see alarming messages.

- **Mobile App Deployment (if applicable):** If you plan to distribute the app via app stores using Capacitor, follow through with building the mobile app (running the provided `setup:mobile` scripts to generate iOS/Android projects). Check that the build is pointing to the production API (Supabase URL) and not a localhost. Test native device functionality like the camera (for QR scanning) and push notifications (if implemented in the future). This may not all be needed for a web pilot, but it's good to have in mind if some users will install the app to their home screen or you give out an APK for testing.

- **Testing on Staging:** Before inviting real users, deploy the application to a staging environment with the above configuration and run through a comprehensive test plan (see below). In particular, test with real devices and with multiple accounts (youth account, guardian email flow, admin account) to cover all perspectives. The staging environment should mirror production settings (just using test data) to catch any environment-specific issues.

By completing this checklist, you ensure that the application's environment is correctly set up and that configuration issues won't derail the pilot launch.

Test Plan for Staging

Before releasing Room XI Connect to real users, it's crucial to verify every major user workflow in a staging environment. Below is a suggested test plan covering both typical user journeys and edge cases. Each item should be tested with different scenarios (e.g. new user vs returning user, minor vs adult, online vs offline):

1. **User Registration & Onboarding:**
2. Create new accounts using various valid emails and passwords. Ensure the sign-up form validations work (e.g. weak password rejection, required fields).
3. Test the email verification process: after sign-up, check that the verification email is received (if using a real email or a testing mail trap) and that clicking the verification link correctly activates the account.

4. For a user under 18, fill in the guardian email and phone. Verify that a guardian verification email is sent and that the system awaits guardian approval (if implemented). Try both scenarios: one where the guardian approves (if that can be simulated or done via a link) and one where they don't, to see how the app behaves (it should possibly restrict access until consent is given).

5. **Login & Authentication Flows:**

6. After verification, log in with the new account. Test that an incorrect password shows the appropriate error and does not log the user in.

7. Use the "Forgot Password" flow: trigger a password reset for an account via the Reset page, check the email, and follow the reset link. On the UpdatePassword page, ensure you can set a new password successfully. After resetting, confirm you can log in with the new password and not with the old one.

8. **Profile Completion & Safety Info:**

9. Once logged in, go to the "Me" or Profile section. Try updating profile fields like name, adding an emergency contact, health info, etc. Ensure that saving works and the data persists after a page refresh.
10. Toggle some consent options (if available in Safety Profile) and ensure these actions are recorded (perhaps check the admin logs as an admin to see that consent changes were logged). If any field is optional, leave it blank and ensure the app handles it gracefully (no crashes or required-field errors unless intended).

11. If the user is under 18, ensure parts of the app that should be restricted or informational (e.g., showing guardian pending status) behave correctly.

12. **Mood Check-In Workflow:**

13. On the Home screen, use the mood check-in feature. Select a mood rating, add some feeling tags or a note, and submit. The UI should update to show today's check-in, and the streak counter should increment appropriately.
14. Do this across multiple days (you can simulate by changing your system date or just test on consecutive days) to verify the streak logic. Also test edge cases like doing multiple check-ins in one day (the app might allow only one per day? If it allows multiple, see how it displays them or if it replaces the previous).

15. Check that mood history graphs or lists update correctly. If there's any sentiment analysis or content filtering (the code includes a "bad-words" dependency – perhaps to filter inappropriate words in notes), try entering some flagged words in the note to see if it's handled (e.g., censored or rejected).

16. **Program Discovery & Saving:**

17. Navigate the Explore page. Test the tab switching between "All Programs", "Map View", and "Saved". Ensure that programs load from the database and are listed with correct information.

18. Use the search or filters if available (e.g., filter by tag or free/paid). Does the list update accordingly? Test the map: are pins displayed at the right locations? Clicking a map pin or a program in the list should open the ProgramDetail page.
19. On the ProgramDetail page, verify all displayed information is populated (given the earlier schema mismatch, see if any fields show up empty or as "undefined"). Try saving a program (click a bookmark icon or save button). The program should then appear under the "Saved" tab or in your Profile. Also verify unsaving (removing from saved).

20. If possible, test with multiple users: one user saves a program and logs out; another user logs in and should not see the first user's saved program (data isolation check). Also confirm that saving programs offline queues properly (turn off network, click save, then restore network and see if the save is recorded in the database).

21. **AI Assistant (Ximi) Behavior:**

22. On the Explore page, interact with the Ximi chat assistant (if it's enabled in the UI – look for an icon, e.g., a chat bubble or a sparkles icon). Click it to open the chat dock.
23. Try asking a few questions or typing messages: for example, "What programs are good for mental health?" or casual conversation. Since the current implementation is rule-based, it may have predefined responses or just a limited understanding. Make sure it responds in a reasonable time and doesn't crash the app.
24. Test a "crisis" message: e.g., mentioning feeling very depressed or something that should trigger the crisis response. The assistant should ideally recognize key phrases and provide a compassionate message plus maybe suggest contacting support or utilizing crisis resources. Check that this logic works as intended.

25. Also test the closing/minimizing of the chat dock and ensure it doesn't interfere with other page elements. If there is an AI API integration active, monitor that no sensitive personal info is sent out in the query.

26. **Attendance Check-In (XID scanning):**

27. If you have the ability to simulate an event QR code: the app likely generates or expects a QR containing an event or location ID. Use the QR scanner feature (usually accessible via a "Scan QR" button or directly from the home or explore page). On a desktop, you might not easily test camera scanning, but you can test the manual code entry if available.
28. For camera testing, use a mobile device or laptop with camera: ensure the camera permission is requested and the scanning works (try generating a QR code corresponding to an expected format if known, or use the device camera on a known QR to see if it at least reads data).
29. If an attendance record is successfully created, verify it shows up in the user's attendance history (maybe on their profile or home). Also check the admin view (if you have admin access) to see if aggregate counts updated.

30. Test the offline scenario: go offline, then attempt to check in (the app might queue the attendance). Then come back online and ensure the queued check-in is synced and appears in the database. This might involve looking at the network tab or console logs for the queue processing.

31. **Admin Dashboard & Permissions:**

32. Log in with an admin account (you may need to designate an account as admin in the database or via the `is_admin` function returning true for your user ID). Verify that the Admin panel becomes accessible (it might be hidden in the UI for non-admins).
33. In the Admin dashboard, check that the statistics (user count, program count, etc.) are displayed and look plausible given your test data.
34. Review the admin logs table: confirm it's pulling recent log entries. If you performed actions like profile updates, consent changes, or program creation (if that exists), those should appear here. The logs should display `old_record` and `new_record` JSON. Verify the content is readable – if not, perhaps this UI could be improved, but at minimum it should not error out.

35. Test that non-admin users cannot access admin routes or see admin content. Try navigating directly to `/admin` as a non-admin; it should redirect or show an unauthorized message. Also ensure that any protected data (like other users' info) truly isn't accessible via the Supabase API thanks to RLS.

36. **Account Deletion:**

37. Using a test account, go to Settings and initiate account deletion. This likely calls the `user-delete` edge function. Confirm that a confirmation step exists (so a user doesn't accidentally delete). Proceed with deletion and ensure that the user is logged out and perhaps redirected to a goodbye or login screen.

38. Verify on the backend that the user's data was actually removed or anonymized. For example, the profile might be wiped (maybe replaced with something like "Deleted User"), mood entries, consents, etc., possibly deleted or dissociated. Also check that the user can't log in again after deletion (the Supabase auth row might be gone or deactivated). This is important for privacy compliance.

39. **Performance and Load:**

    - While a full load test might be out of scope for staging, try to simulate a bit of stress: create, say, 10-20 users (perhaps via script or manually) and have them perform actions. Ensure the app remains responsive.
    - Check the client performance on low-end devices or throttled network: does the app load within a few seconds on 3G speeds? Are images optimized and is bundle size reasonable? Since this is a React app, ensure code splitting is in place for large libraries (leaflet map, etc., should not block initial load of other pages).
    - Also, keep an eye on memory usage (especially if the app is left open in the browser for a long time, does it bloat?). Any memory leaks in event listeners or intervals (for example, the offline queue retry mechanism or any real-time listeners) should be identified and fixed.

Throughout testing, document any new issues encountered and fix them prior to pilot. It's helpful to have a checklist and mark each scenario as pass/fail with notes, so nothing is overlooked. Engaging a few test users (or colleagues) to run through this plan can bring fresh eyes and catch issues you might miss.

Conclusion

Overall, the Room XI Connect app is a well-structured and feature-rich application tailored for youth engagement. The code review process surfaced a few critical bugs (now fixed, such as the admin authorization logic, attendance querying, and password reset flow) and highlighted several areas for improvement. Most features – registration, mood tracking, program discovery, safety profile, and offline support – are implemented and working, with just some enhancements needed for a polished experience (like dark mode, full data export, and completion of the AI assistant and notification features).

By addressing the issues and recommendations above, and by thoroughly testing each workflow in a staging environment, the app should be ready for a smooth public pilot. The final pre-pilot checklist is to ensure all configuration is correct (Supabase, environment variables, service worker, etc.) and that no major bugs or UX issues remain. Once that's done, you can confidently launch the pilot, knowing that the application has been audited for security, privacy compliance, and reliability. With real user feedback during the pilot, you can then iterate on any minor tweaks.

Ready for Pilot: With the critical fixes applied and the remaining items either resolved or noted, Room XI Connect should be issue-free enough for real users in a pilot setting. It's an exciting step to see it in use – good luck, and be prepared to support users as they onboard and use the app in the real world!
